

SPEC- 1 -AI Governance Engine

Background

Organizations are rapidly deploying AI/ML systems (including LLM-powered workflows) across products and internal operations. This creates new governance needs that are difficult to manage with ad-hoc spreadsheets and scattered documents:

- A single inventory of AI systems, models, datasets, vendors, and deployments.
- Evidence-driven controls (policies, procedures, technical safeguards) mapped to external frameworks (e.g., NIST AI RMF) and management standards (e.g., ISO/IEC 42001).
- Risk classification, approvals, and change control across the AI lifecycle.
- Continuous monitoring (drift, incidents, security events, human oversight) with auditable logs.
- Reporting for auditors, regulators, leadership, and engineering teams.

This spec describes **AI Governance Engine** a system that centralizes AI governance data, runs assessments/scoring, manages evidence and approvals, and produces compliance-ready reports and remediation roadmaps.

Requirements

Must have

- **AI inventory:** register AI systems, models, deployments, owners, intended use, jurisdictions, and vendors.
- **Control library:** configurable governance controls grouped by category (policy, data, model, security, monitoring, human oversight, etc.).
- **Assessments:** run structured assessments per AI system (questionnaires + evidence links) producing:
 - per-control status (Missing/Partial/Provided/Verified/NA)
 - maturity scoring and readiness/risk tier
 - critical flags & missing elements
- **Evidence management:** store references to evidence (documents, tickets, URLs) and tie them to controls and assessment runs.
- **Remediation roadmap:** generate prioritized actions (who/what/when) from gaps.
- **Auditability:** immutable audit log for changes, decisions, and approvals.
- **Role-based access control:** at minimum Admin, Governance Lead, Assessor, System Owner, Read-only.
- **Report outputs:** exportable artifacts (PDF/HTML/JSON) for audit packages and executive summaries.

Should have

- **Framework mapping:** map controls to multiple frameworks/regs (e.g., NIST AI RMF, ISO/IEC 42001, EU AI Act obligations) via crosswalk tables.

- **Workflow:** review/approve gates (e.g., before production, model update, new data source).
- **Integrations:** connectors for common systems (Jira/Linear, GitHub, model registries, cloud logs).
- **Continuous monitoring hooks:** ingest monitoring signals (incidents, drift metrics, eval results) and raise governance alerts.

Could have

- **Policy-as-code:** declarative rules that automatically flag noncompliance (e.g., missing DPIA/impact assessment for high-risk use).
- **Shadow AI discovery:** lightweight discovery signals (SSO logs, proxy, SaaS usage) feeding an intake workflow.
- **Multi-tenant:** support multiple business units/clients with strict tenant isolation.

Won't have (MVP)

- Full automated regulatory filing/CE marking end-to-end.
- Real-time enforcement proxy in front of every inference call (we'll provide hooks, not a mandatory gateway).

Method

Scope for your use case (Consultant product)

Assumption (based on your answers): you'll use **repeatable product** across multiple client organizations, and the MVP should integrate first with **model tooling / registries**.

High-level architecture

- **Web App (UI):** multi-tenant governance workspace (inventory, assessments, evidence, reports).
- **Governance API:** CRUD + workflows + reporting.
- **Scoring Engine:** deterministic rules that compute control status, maturity, and roadmap items.
- **Connector Workers:** pull model metadata, deployments, evaluations, lineage from client tooling.
- **Object Store:** evidence attachments + report bundles.

Recommended MVP stack (contractor-friendly):
API: -FastAPI + Pydantic v 2 (aligns with your existing `aigov-builder` package)
DB: PostgreSQL (multi-tenant `tenant_id` + Row Level Security option) -
Async: Celery or RQ + **Auth:** OIDC (Auth 0/Okta/Azure AD) + **UI:** React (or Next.js) + component library

Multi-tenancy model

Two supported modes (choose per client):
 1) **Shared DB + shared schema** (`tenant_id` on every table (lowest cost, fastest).
 2) **Dedicated DB per tenant** (premium isolation for regulated clients).

MVP: implement **shared DB** with strict application-layer enforcement; optional **Postgres RLS policies** once stable.

Core components (PlantUML)

```
@startuml
skinparam componentStyle rectangle

package "Client Org" {
    [User Browser] as browser
    [Client IdP (OIDC)] as idp
    [Model Tooling] as tools
    [Model Registry] as registry
    [Monitoring/Evals] as monitor
}

package "AI Governance Engine (SaaS)" {
    [Web UI] as ui
    [Governance API] as api
    [Scoring Engine
(aigov-builder)] as engine
    [Connector Worker] as worker
    [Report Builder] as reports
    database "PostgreSQL" as db
    [Object Store] as blob
    [Event Bus/Queue] as queue
    [Audit Log Writer] as audit
}

browser --> ui
ui --> api
api --> idp
api --> db
api --> blob
api --> engine
api --> reports
api --> audit

worker --> queue
api --> queue
worker --> tools
worker --> registry
worker --> monitor
worker --> db

reports --> db
reports --> blob
@enduml
```

Data model (MVP tables)

All tables include `tenant_id` (UUID) + `created_at`, `updated_at`.

Tenancy & identity `tenants(id, name, plan, region, settings_json)` `users(id, tenant_id, email, display_name, status)` - `roles(id, tenant_id, name)` - `user_roles(user_id, role_id)`

Inventory `ai_systems(id, tenant_id, name, description, business_owner_user_id, tech_owner_user_id, lifecycle_stage, jurisdictions_json, risk_tier, status)` - `models(id, tenant_id, ai_system_id, name, type, provider, registry_ref, version, hash, training_data_ref, eval_summary_json)` `deployments(id, tenant_id, model_id, env, endpoint, region, purpose, monitoring_ref, last_seen_at)` `datasets(id, tenant_id, name, kind, location_ref, pii_level, lineage_ref, retention_policy_ref)` - `vendors(id, tenant_id, name, type, contract_ref, dpia_ref)`

Controls & frameworks `control_sets(id, tenant_id, name, version, description)` - `controls(id, tenant_id, control_set_id, code, title, description, category, severity, evidence_types_json, applicability_rules_json)` - `frameworks(id, tenant_id, name, version)` `framework_requirements(id, tenant_id, framework_id, code, title)` - `control_mappings(id, tenant_id, control_id, framework_requirement_id, mapping_strength)`

Assessments & evidence `assessments(id, tenant_id, ai_system_id, control_set_id, status, started_by_user_id, completed_at)` - `assessment_items(id, tenant_id, assessment_id, control_id, status, score, notes, verified_by_user_id, verified_at)` - `evidence(id, tenant_id, type, title, uri, blob_key, hash, source_system, source_ref, collected_at)` - `assessment_evidence(id, tenant_id, assessment_item_id, evidence_id)`

Workflow & audit - `approvals(id, tenant_id, ai_system_id, gate, status, requested_by_user_id, decided_by_user_id, decided_at, rationale)` - `change_requests(id, tenant_id, ai_system_id, kind, status, payload_json, created_by_user_id)` - `audit_events(id, tenant_id, actor_user_id, action, entity_type, entity_id, before_json, after_json, occurred_at)`

Roadmap - `roadmap_items(id, tenant_id, ai_system_id, control_id, priority, owner_user_id, due_date, status, rationale, generated_from_assessment_id)`

Scoring & roadmap algorithm

Use a deterministic, auditable rules engine (build on your existing `aigov-builder` package):

1) **Applicability**: each control `applicability_rules_json` evaluated against the AI system (e.g., jurisdiction includes EU, model type is LLM, dataset ~~Status PII~~ **points** map status to points (e.g., Missing= 0, Partial= 0.5, Provided= 0.8, Verified= 1.0, NA exclude **Category**) **rollups**

weighted average by category and severity. **Risk tier adjustments** high-risk systems require more controls. Verified to pass gates. **Roadmap generation** for each Missing/Partial item, emit a recommended action template with severity-based priority and due date SLA.

Sequence (assessment run):

```
@startuml
actor Assessor
participant UI
participant API
participant Engine
database DB

Assessor -> UI: Start Assessment
UI -> API: POST /assessments
API -> DB: create assessment + items
API -> Engine: compute initial scoring
Engine -> DB: write scores/status rollups

Assessor -> UI: Attach Evidence
UI -> API: POST /evidence + link
API -> DB: store evidence + links

Assessor -> UI: Submit for Verification
UI -> API: POST /assessments/{id}/submit
API -> Engine: recompute
Engine -> DB: update rollups
API -> DB: set status=Submitted
@enduml
```

Model tooling integrations (MVP)

Goal: automatically populate inventory and keep it fresh.

Connector pattern: - Each connector ~~job~~ **data** runs on a schedule or webhook. - Normalizes tool-specific metadata into a common model **registry_ref** (model URI), version, tags, metrics, artifacts, lineage pointers.

Suggested first-class connectors: **MLflow Tracking/Registry** (common across many clients; also used by Databricks) **AWS SageMaker Model Registry** **Model Package Group** **Google Vertex AI Model Registry** - **Azure ML registry / MLflow-backed registry**

MVP connector outputs: - Create/update **models**, **deployments**. - Store tool provenance in **source_system** + **source_ref**. - Optionally ingest evaluation summaries into **eval_summary_json**.

Security & compliance design

- **Tenant isolation:** enforce `tenant_id` on all queries; optionally enable Postgres RLS per table.
- **RBAC:** Admin / Governance Lead / Assessor / System Owner / Read-only (extendable).
- **Audit log:** append-only `audit_events` with before/after snapshots.
- **Evidence integrity:** store hashes; immutable object-store keys; signed URLs for access.

Similar systems (for inspiration / parity targets)

- **MLflow** (model registry/metadata), **SageMaker/Vertex/Azure ML registries** (model lifecycle), and governance GRC platforms (controls/evidence workflows). The differentiator here is **AI-specific crosswalking + scoring + lifecycle gates** tied directly to model tooling and deployments.

Implementation

MVP delivery shape

Deliver as two deployable artifacts (**SaaS**):
1) **Control Plane** (your product): UI + API + DB + reporting tenant management.
2) **Client Connector Agent** (per customer): deploys in their cloud/VPC, pulls model/tooling metadata (MLflow first) and pushes normalized snapshots/events to the control plane.

Repos / packages

- Reuse your existing Python package in this repo: `aigov-builder` (scoring engine + reporting utilities).
- Add two top-level services:
 - `gov_api/` (FastAPI service)
 - `gov_ui/` (Next.js UI)
 - `connector_agent/` (Python agent + connector plugins)

Control Plane: API (FastAPI)

Key API surfaces (MVP):
- **Auth** - `POST /auth/exchange` (OIDC token → session/JWT)
- **Middleware:** enforce `tenant_id` from token claim
- **Inventory** - `GET/POST /ai-systems` - `GET/POST /models` - `GET/POST /deployments` - `GET/POST /datasets` - `GET/POST /vendors`
- **Controls** - `GET/POST /control-sets` - `GET/POST /controls` - `GET/POST /frameworks` - `POST /controls/{id}/mappings`
- **Assessments** - `POST /ai-systems/{id}/assessments` (creates items for applicable controls) - `PATCH /assessment-items/{id}` (status, notes, verification) - `POST /assessments/{id}/submit` - `POST /assessments/{id}/recompute` (invokes scoring engine)
- **Evidence** - `POST /evidence` (metadata + upload initiated) - `POST /assessment-items/{id}/evidence/{evidence_id}` (link)
- Evidence uploads via pre-signed URL to **Object Store**
- `GET /ai-systems/{id}/roadmap` - `POST /roadmap-items/{id}/assign` - `PATCH /roadmap-items/{id}` (status updates)
- **Reporting** - `POST /ai-systems/{id}/reports` (bundle outputs to object store) - `GET /reports/{id}/download`
- **Audit** - `GET /audit-events` (filtered, read-only)

Control Plane: persistence & migrations

- PostgreSQL with migrations using Alembic.
- Enforce `tenant_id` on all tables.
- Add unique constraints:
 - `(tenant_id, ai_systems.name)`
 - `(tenant_id, controls.control_set_id, controls.code)`
 - `(tenant_id, frameworks.name, frameworks.version)`
- Add indexes:
 - `(tenant_id, updated_at)` on most tables
 - `(tenant_id, ai_system_id)` on assessment/roadmap tables

Scoring integration (reuse `aigov-builder`)

- Implement a thin adapter layer:
 - `domain -> aigov_builder` mapping (controls, assessment items, evidence links)
 - `aigov_builder outputs -> DB rollups + roadmap_items`
- Recompute triggers:
 - on assessment item update
 - on evidence link/unlink
 - on control set version change

Client Connector Agent (MLflow-first)

Goal: avoid pulling client credentials into your SaaS.

Agent characteristics: - Runs as Docker/K8s deployment inside client environment. - Uses client's network access to reach MLflow tracking/registry. - Periodically publishes: - model registry state (version, tags) - latest eval metrics (where available) - deployment references (if known)

Control plane endpoint: - `POST /ingest/model-snapshots` (signed client token, tenant-bound)

Security: - **Agent key per tenant** with scoping to ingest endpoints only. - Payload signing (HMAC) replay protection (timestamp + nonce).

MLflow mapping (typical): - Model name/v `models.name`, `models.version` - Run ID / artifact URI → `models.registry_ref` - Tags/metrics → `models.eval_summary_json`

Evidence storage

- Object store: S3 (or compatible) with tenant-prefixed keys: `tenant/{tenant_id}/evidence/{evidence_id}`.
- Store `hash` and `collected_at` in `evidence` table.
- Pre-signed URLs for upload/download; never proxy raw files through API.

UI (Next.js)

MVP screens: - Tenant switcher (for you as consultant) + client admin basics. - AI Systems list (inventory + model/deployment tabs). - Assessment runner (control checklist, evidence attachments, verification). - Roadmap Kanban/table (priority, owner, due date). - Reports page (generate + download bundle).

Deployment blueprint

- **SaaS:** containerized services (API + worker + UI) + Postgres + Redis + object store.
- **Environments:** dev/stage/prod with separate DBs.
- **Observability:** structured logs + metrics + traces.

Hardening (post-MVP but planned)

- Optional Postgres RLS policies per table.
- Dedicated DB-per-tenant option for premium clients.
- Connector plugins for SageMaker/Vertex/Azure ML.

Milestones

M 0 — Product framing (1 – 3 days)

- Define tenant onboarding workflow (how you create a client tenant, invite users, connect agent).
- Finalize MVP control set(s) and assessment rubric.

M 1 — Control plane foundation

- Postgres schema + Alembic migrations.
- FastAPI service skeleton with OIDC auth + RBAC.
- Tenant-scoped CRUD for AI systems, models, deployments.
- Audit event writer for all state changes.

M 2 — Controls + assessments

- Control set CRUD + applicability rules.
- Assessment creation (generate items from applicable controls).
- Assessment UI (checklist + notes + status workflow).

M 3 — Evidence + scoring + roadmap

- Evidence storage (pre-signed uploads) + linking.
- Integrate `aigov-builder` recompute pipeline.
- Roadmap item generation + UI view.

M 4 — MLflow connector agent (MVP)

- Agent packaging (Docker) + secure tenant-bound key.

- MLflow connector plugin (snapshot → ingest API).
- Inventory auto-refresh from snapshots.

M 5 — Reporting & audit package

- Executive summary report + detailed control matrix.
- Export bundle to object store (PDF/HTML/JSON).
- Governance gate: “Ready for production” decision record.

M 6 — Pilot with 1 – 2 client orgs

- Onboard first tenants.
- Tune control applicability rules + roadmap templates.
- Collect feedback, harden auth/tenancy, improve UX.

Gathering Results

Success criteria (MVP)

- **Inventory completeness:** % of client AI systems/models tracked vs known sources.
- **Assessment throughput:** time to complete an assessment for one AI system; number of controls verified.
- **Evidence quality:** % of controls with evidence; % with integrity hash; verification rate.
- **Roadmap effectiveness:** reduction in Missing/Partial controls between assessment runs.
- **Audit readiness:** ability to produce an audit package within hours (not weeks).

Operational metrics

- Connector health: last snapshot time, snapshot failures, connector coverage.
- API latency and error rate.
- Report generation time and failure rate.

Post-production evaluation

- Run quarterly governance reviews:
 - changes in control set versions
 - drift in risk tiers across systems
 - incident/monitoring signals mapped to governance alerts
- Gather client feedback on:
 - clarity of controls and recommended actions
 - usability of evidence workflow
 - integration coverage (next connectors)

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)